Department of CSE, PES University

COMPUTER NETWORKS-UE24CS252B

COMPUTER NETWORKS - SDN MININET PROJECT

SDN-BASED QOS TRAFFIC ANALYSIS USING MININET AND POX

1. INTRODUCTION

Software Defined Networking (SDN) is an advanced networking paradigm that separates the control plane from the data plane, enabling centralized control of network behavior. In this project, an SDN-based network is implemented using Mininet and the POX controller to analyze Quality of Service (QoS) behavior under different traffic conditions.

The objective of this project is to simulate different types of network traffic (ICMP, HTTP, and bulk TCP traffic) and observe their behavior through a bottleneck link using Wireshark analysis.

2. OBJECTIVES

- To design an SDN topology using Mininet
- To implement a POX controller for traffic classification
- To generate different types of traffic (ICMP, HTTP, iperf)
- To analyze network behavior under single and mixed traffic scenarios
- To capture and verify packet traversal using Wireshark

3. TOOLS AND TECHNOLOGIES USED

- **Mininet** – for network emulation
- **POX Controller** – for SDN control logic
- **Python** – for controller implementation
- **Wireshark** – for packet analysis
- **iperf** – for generating bulk TCP traffic
- **HTTP Server (Python)** – for web traffic generation

4. NETWORK TOPOLOGY

The following topology is implemented:

```
h1     h2
 |      |
  \    /
   s1 -------- s2 -------- s3
              |         |
             h3        h4
```

Description:

- s1 connected to h1 and h2
- s2 connected to h3 (bottleneck switch)
- s3 connected to h4
- Inter-switch links: s1 ↔ s2 ↔ s3
- Link bandwidth: **10 Mbps**

Switch s2 acts as the **bottleneck point** where traffic contention is observed.

5. IMPLEMENTATION STEPS

5.1 SETUP AND INITIALIZATION

- Installed Mininet and POX controller
- Created custom topology using Python
- Configured remote controller connection
- Verified connectivity using: pingall

5.2 CONTROLLER IMPLEMENTATION

A custom QoS controller is implemented using POX which:

- Listens to PacketIn events
- Identifies traffic types:
  - ICMP → High Priority
  - HTTP (port 8080) → Medium Priority
  - TCP (port 5001) → Low Priority
- Logs traffic classification

5.3 TRAFFIC GENERATION

Three types of traffic are generated:

1. ICMP Traffic: h1 ping h4

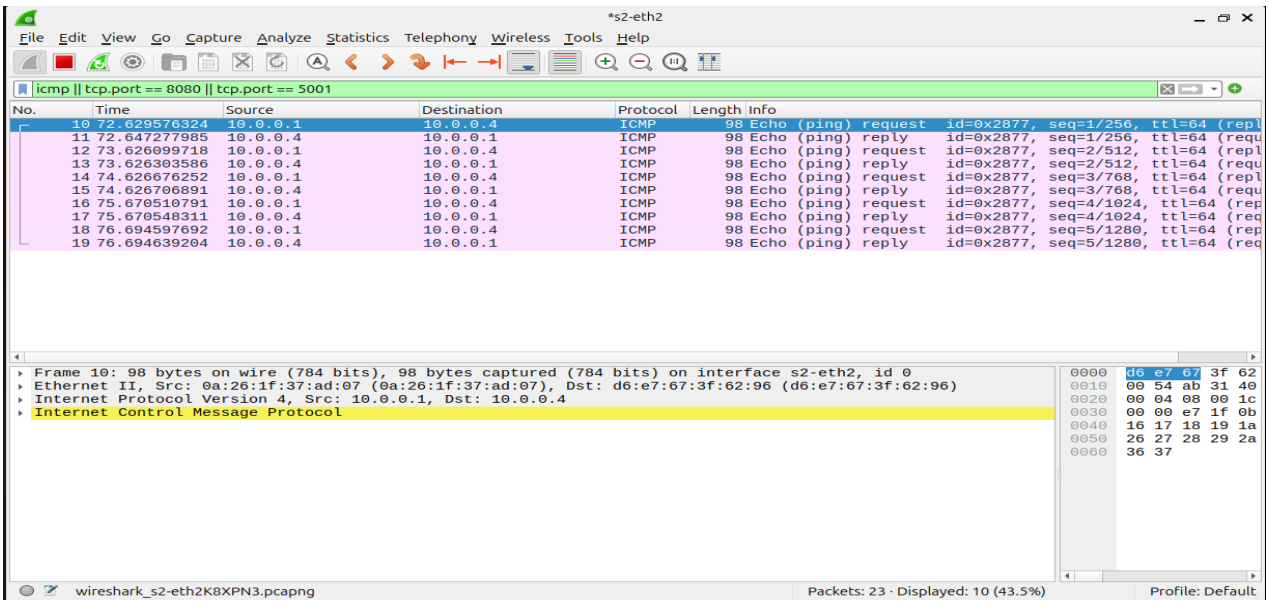
2. HTTP Traffic: h2 python3 -m http.server 8080
h4 curl http://10.0.0.2:8080

3. Bulk TCP Traffic (iperf): h3 iperf -s -p 5001
h1 iperf -c 10.0.0.3 -p 5001

6. EXPERIMENTAL RESULTS AND ANALYSIS

6.1 ICMP TRAFFIC ANALYSIS (HIGH PRIORITY)

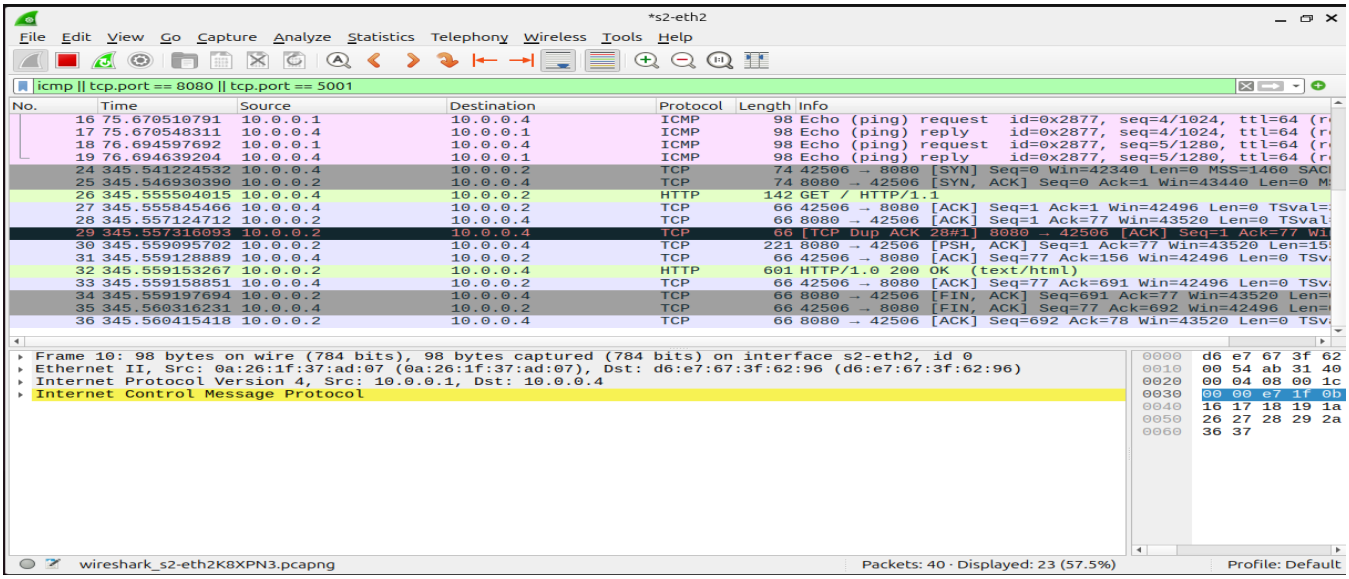
wireshark_icmp.png



ICMP packets are generated using the ping command between hosts h1 and h4. Wireshark capture shows Echo Request and Reply packets. This traffic is considered high priority as it is latency-sensitive.

6.2 HTTP TRAFFIC ANALYSIS (MEDIUM PRIORITY)

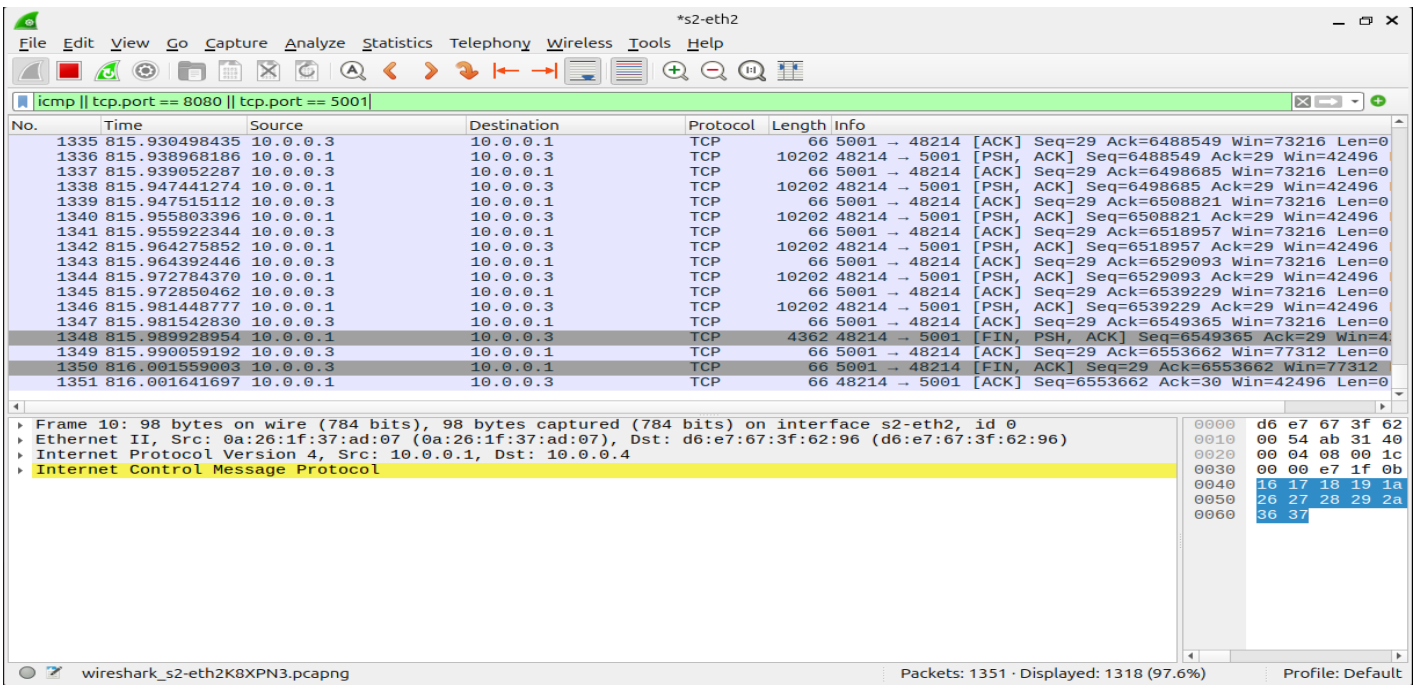
wireshark_http_8080.png:



HTTP traffic is generated using a Python web server.
Wireshark shows TCP handshake followed by HTTP GET request and response (200 OK). This represents medium-priority traffic.

6.3 BULK TRAFFIC ANALYSIS USING IPERF (LOW PRIORITY)

wireshark_iperf_5001.png:



Bulk TCP traffic is generated using iperf. Continuous TCP packets with PSH and ACK flags are observed, indicating high throughput data transfer. This is considered low-priority traffic as it is not delay-sensitive.

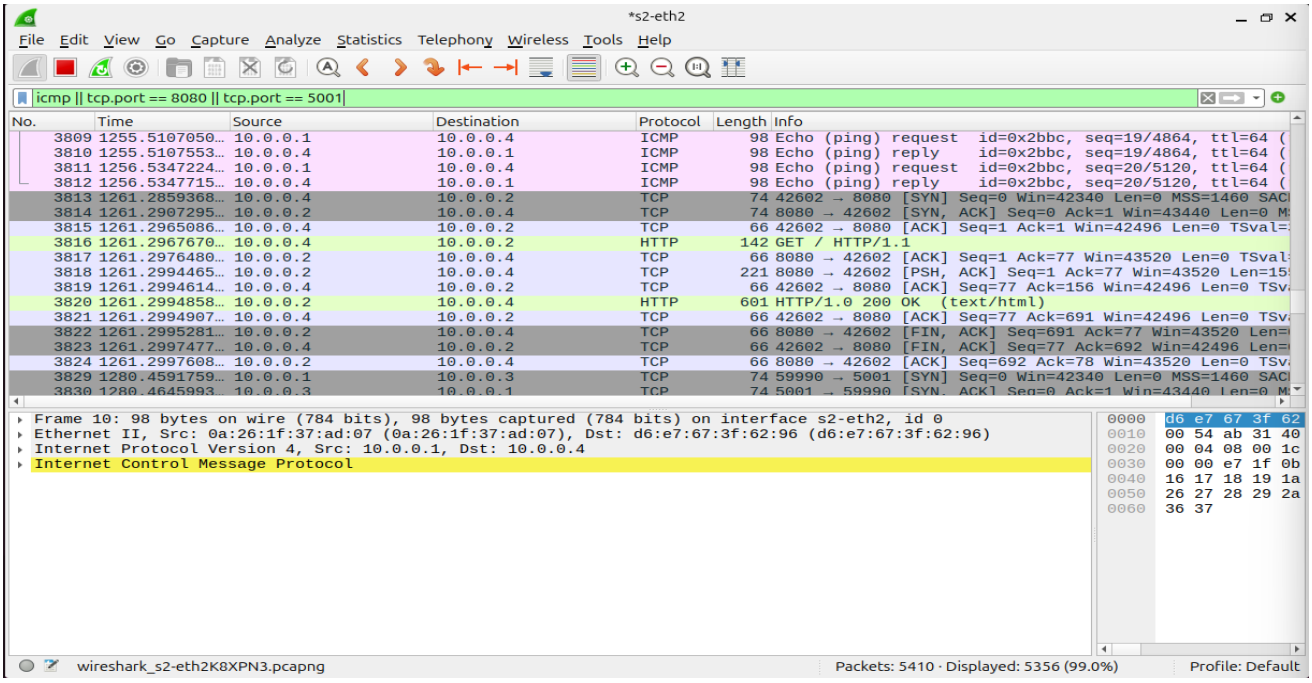
6.4 Mixed Traffic Analysis (All Traffic Types)

All three traffic types are generated simultaneously:

- ICMP (ping)
- HTTP (port 8080)
- TCP bulk traffic (port 5001)

Wireshark capture shows all packets coexisting at the bottleneck switch (s2-eth2). This demonstrates real-world network behavior under congestion conditions.

wireshark_all_traffic.png:



7. OBSERVATIONS

- ICMP packets are small and frequent, representing latency-sensitive traffic
- HTTP traffic shows request-response behavior with moderate bandwidth usage
- iperf traffic generates continuous bulk data, consuming maximum bandwidth
- When all traffic types are combined, congestion occurs at the bottleneck switch
- Wireshark confirms simultaneous packet traversal

8. CONCLUSION

This project successfully demonstrates the behavior of different traffic types in an SDN-based network. By implementing a custom topology and controller, and generating multiple traffic scenarios, it is observed that different types of traffic coexist and compete for bandwidth at the bottleneck link.

The use of Wireshark provides clear evidence of packet-level interactions, validating the functionality of the SDN-based QoS model. The experiment highlights the importance of traffic classification and management in modern networks.

9. FUTURE SCOPE

- Implement actual flow rule prioritization using OpenFlow
- Introduce queue-based scheduling (QoS queues)
- Use advanced controllers like Ryu
- Analyze delay, jitter, and packet loss quantitatively

THANK YOU